

App Dev Project Report

1. Student Details

- Name: raj dhakar
- Roll Number: 23f3002897

2. Project Details

- Project Title: Trekking Management Application (TMA)
- Problem Statement:

To design and build a multi-role, full-stack web application that streamlines the management of trekking routes, guide assignments, and participant bookings. The platform must enforce strict booking constraints (overbooking prevention and duplicate check checks), role-based access control (Admin, Trek Staff Guide, and Trekker), and automated background job processing for daily reminders and monthly operational reports.

- Approach:

The application is architected using a decoupled Single Page Application (SPA) approach. The backend is built on Python Flask with SQLAlchemy ORM and SQLite for persistent data storage, integrated with Redis for caching high-frequency public endpoints and Celery for asynchronous background job execution. The frontend is built using Vue 3 and Bootstrap 5, communicating with the REST API via Axios and securing sessions with JSON Web Tokens (JWT).

3. AI/LLM Declaration

I used LLM assistants to assist in writing SQLAlchemy model definitions and debugging complex code logic across backend routes.

The extent of AI/LLM usage is around 5–10%, strictly limited to initial code structuring suggestions and debugging assistance.

All core business logic, role-based access control, frontend Single Page Application workflows, database relationships, and Celery asynchronous batch job integrations were designed, written, and finalized manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web REST API framework (backend/app.py)
SQLAlchemy / Flask-SQLAlchemy	Object Relational Mapper (ORM) for database interactions (backend/models.py)
SQLite	Lightweight relational database for persistent storage (backend/tma.db)
Vue 3 + Vite	Modern reactive frontend Single Page Application framework (frontend/src/)

Vue Router 4	Frontend routing and role-based navigation guards (frontend/src/router/index.js)
Bootstrap 5	Responsive UI styling, grid layouts, navigation, and modal components
PyJWT	JSON Web Token authentication and Role-Based Access Control (@token_required, @role_required)
Redis / Flask-Caching	In-memory caching for frequently accessed trek listings with automatic expiration
Celery	Asynchronous task queue and worker execution engine for batch processing
Celery Beat	Periodic cron job scheduler for daily reminders and monthly reports

5. Database Schema / ER Diagram

Tables & Attributes:

1. User

- Stores account authentication and profile information across all roles.
- Attributes: id (PK), email (Unique), password_hash, full_name, role ('admin', 'staff', 'user'), phone, emergency_contact, is_active (Boolean).

2. StaffProfile

- Stores professional certifications and experience details for Trek Staff Guides.
- Attributes: id (PK), user_id (FK -> user.id), experience_years, bio.

3. Trek

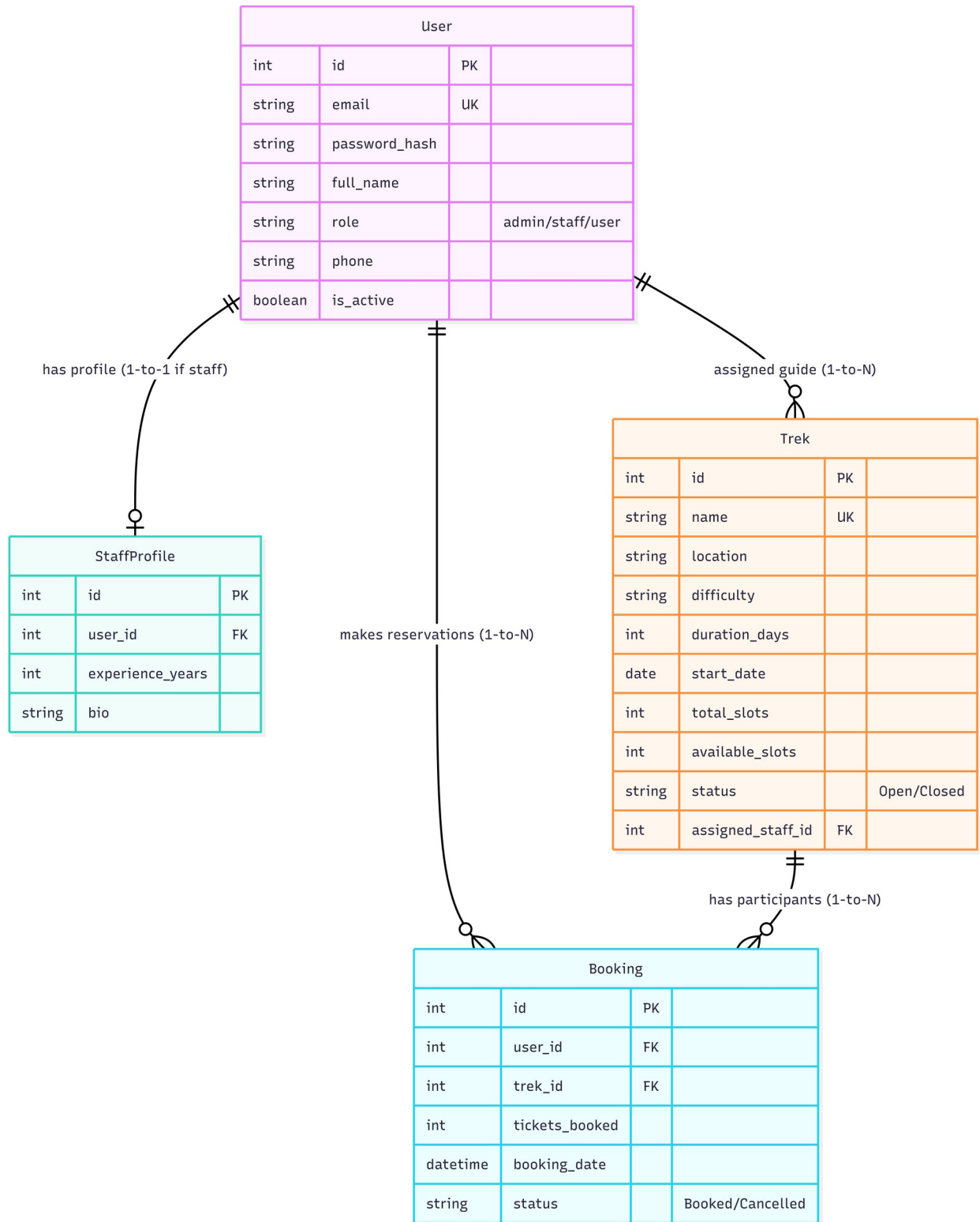
- Stores comprehensive trekking route details, dates, capacity, and status.
- Attributes: id (PK), name, location, difficulty ('Easy', 'Moderate', 'Hard'), duration_days, start_date, end_date, total_slots, available_slots, description, status ('Open', 'Closed', 'Completed'), assigned_staff_id (FK -> user.id).

4. Booking

- Stores individual trekker bookings, ticket counts, timestamps, and reservation status.
- Attributes: id (PK), user_id (FK -> user.id), trek_id (FK -> trek.id), tickets_booked, booking_date, status ('Booked', 'Cancelled', 'Completed').

Relationships:

- One-to-One: User (Role: staff) -> StaffProfile
- One-to-Many: User (Role: staff) -> Trek (Assigned Treks)
- One-to-Many: User (Role: user) -> Booking (Trekker Reservations)
- One-to-Many: Trek -> Booking (Route Participants)



6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/register	POST	Register a new trekker account
/api/auth/login	POST	Authenticate user credentials and return JWT token
/api/auth/profile	GET / PUT	Fetch or update personal profile details and password
/api/treks	GET	Fetch all public treks (Cached via Redis for 300 seconds)
/api/treks/<id>/book	POST	Book tickets for a trek with strict overbooking & duplicate validation
/api/my-bookings	GET	Fetch complete historical and active bookings for the logged-in trekker
/api/my-bookings/export-csv	POST	Trigger asynchronous Celery batch job to export booking history as CSV
/api/admin/dashboard	GET	Fetch system-wide analytics, counts, and charts for Admin
/api/admin/treks	GET / POST	Admin management: List all or create a new trekking route
/api/admin/treks/<id>	PUT / DELETE	Admin management: Update or delete a specific trekking route
/api/admin/staff	GET / POST	List all staff members or register a new staff guide account
/api/admin/users	GET	List all registered trekkers with booking activity summary
/api/admin/users/<id>/toggle-active	PUT	Blacklist or reactivate a user account (is_active toggle)
/api/admin/tasks/trigger-reminders	POST	Manually trigger Daily Reminders Celery task on demand
/api/admin/tasks/trigger-monthly-report	POST	Manually trigger Monthly Activity Report Celery task on demand
/api/staff/treks	GET	Fetch treks assigned specifically to the logged-in staff member
/api/staff/treks/<id>	PUT	Update trek status (Open/Closed/Completed)

		and available_slots
/api/staff/treks/<id>/participants	GET	View list of registered trekkers for the staff member's assigned trek

7. Architecture and Features

Architecture Overview:

- backend/app.py – Main Flask application entry point and factory.
- backend/config.py & extensions.py – Environment settings, SQLAlchemy, Redis, and Celery initialization.
- backend/models.py – Database ORM models and relationships.
- backend/routes/ – Modular Flask Blueprints (auth.py, admin.py, staff.py, user.py).
- backend/tasks.py & celery_worker.py – Celery asynchronous batch jobs and Beat cron schedule.
- frontend/src/views/ – Vue 3 Single Page Application dashboards for each role.

Implemented Features:

- Role-Based Access Control: Distinct portals with strict navigation and API security for Admin, Staff Guide, and Trekker.
- Strict Business Logic Validations:
 - Prevents overbooking beyond available_slots using atomic slot adjustments.
 - Ensures only assigned staff guides can update or manage their specific trek routes.
 - Allows ticket booking strictly when trek status is 'Open'.
 - Prevents duplicate active bookings for the same trek by the same user.
- Account Blacklisting: Admin can deactivate user accounts; deactivated users are blocked system-wide.
- High-Performance Caching: Redis caching on public trek exploration (GET /api/treks) with instant cache invalidation upon route modifications.
- Asynchronous Batch Processing (Celery):
- Scheduled Job A (Daily Reminders): Runs daily at 8:00 AM to scan for treks starting in 3 days; logs notifications and sends emails/webhooks.
- Scheduled Job B (Monthly Activity Report): Runs on the 1st of every month to generate a comprehensive HTML summary report for the Admin (backend/generated_reports/).
- User Triggered Job C (CSV Export): Generates a downloadable CSV of historical user bookings asynchronously (backend/static/exports/).

8. Video Presentation

- Google Drive Video Link:
<https://drive.google.com/drive/folders/1jQVaJmf6ranFVFzsPBwZudXpMaw9Nxfr?usp=sharing>